

SUMMARY

HelixCode User Manual - Creation Summary

Date: November 6, 2025 **Version:** 2.0 **Author:** Claude AI via HelixCode

Overview

A comprehensive user manual for HelixCode has been created with complete documentation covering all features, providers, tools, workflows, and advanced use cases.

Directory Structure Created

```
/Users/milosvasic/Projects/helix_code/docs/user_manual/
├── README.md                    # Main comprehensive manual (3,240
├── SUMMARY.md                  # This file
├── chapters/                   # Reserved for future chapter-based
├── images/                     # Reserved for diagrams and screens
├── tutorials/                  # Step-by-step tutorials (10 tutori
    ├── Tutorial_1_Building_Web_App.md      (1,245 lines)
    ├── Tutorial_2_Refactoring_Large_Codebase.md (133 lines)
    ├── Tutorial_3_Multiple_AI_Providers.md  (110 lines)
    ├── Tutorial_4_Browser_Automation.md     (149 lines)
    ├── Tutorial_5_Voice_to_Code.md          (118 lines)
    ├── Tutorial_6_Multi_File_Atomic_Edits.md (134 lines)
    ├── Tutorial_7_Distributed_Development.md (165 lines)
    ├── Tutorial_8_Using_Plan_Mode.md         (301 lines)
    ├── Tutorial_9_Custom_Provider.md         (79 lines)
    └── Tutorial_10_Adding_a_Tool.md          (80 lines)
├── examples/                   # Configuration examples (4 files)
    ├── basic_config.yaml       (57 lines)
    ├── provider_configs.yaml   (174 lines)
    ├── enterprise_setup.yaml   (248 lines)
    └── multi_worker_setup.yaml (261 lines)
```

File Statistics

Main Documentation

File	Lines	Description
README.md	3,240	Comprehensive user manual covering all features

Tutorials (10 files)

Tutorial	Lines	Duration	Level
Tutorial 1: Building Web App	1,245	45-60 min	Beginner
Tutorial 2: Refactoring	133	30-45 min	Intermediate
Tutorial 3: Multiple AI Providers	110	20 min	Beginner-Intermediate
Tutorial 4: Browser Automation	149	30 min	Intermediate
Tutorial 5: Voice-to-Code	118	15 min	Beginner
Tutorial 6: Multi-File Edits	134	20 min	Intermediate
Tutorial 7: Distributed Development	165	30 min	Advanced
Tutorial 8: Plan Mode	301	45 min	Intermediate-Advanced
Tutorial 9: Custom LLM Provider	79	30 min	Advanced
Tutorial 10: Tool via MCP SDK	80	30 min	Advanced
TOTAL	2,514		

Configuration Examples (4 files)

Example	Lines	Purpose
basic_config.yaml	57	Development and small projects
provider_configs.yaml	174	All 14+ LLM provider configurations
enterprise_setup.yaml	248	Production enterprise deployment
multi_worker_setup.yaml	261	Distributed worker orchestration
TOTAL	740	

Grand Total

6,494 lines of comprehensive documentation

Main README.md Contents

Part I: Getting Started

- 1. **Introduction** (4 sections)
 - What is HelixCode?

- Key Features
- Architecture Overview
- Use Cases
- 2. **Installation** (6 sections)
 - System Requirements
 - Quick Start Installation
 - Production Deployment
 - Docker Deployment
 - Kubernetes Deployment
 - Platform-Specific Setup (Linux, macOS, Windows)
- 3. **First Project** (3 sections)
 - Creating Your First Project
 - Basic Workflow
 - Understanding Sessions

Part II: Configuration

1. **Configuration** (4 sections)
 - YAML Configuration (complete config.yaml with all options)
 - Environment Variables (50+ variables documented)
 - Configuration Profiles (dev, staging, production)
 - Security Best Practices

Part III: LLM Providers (14+ Providers)

1. **LLM Providers Overview** (3 sections)
 - Provider Selection Guide
 - Cost Comparison Table
 - Performance Comparison
2. **Premium Providers** (8 providers)
 - **Anthropic Claude**: Extended thinking, prompt caching, 200K context
 - **Google Gemini**: 2M token context, multimodal, flash models
 - **OpenAI**: GPT-4.1, GPT-4o, O1/O3 reasoning models
 - **AWS Bedrock**: Multi-model enterprise platform
 - **Azure OpenAI**: Entra ID, enterprise compliance
 - **Google VertexAI**: Unified AI platform
 - **Groq**: Ultra-fast 500+ tok/s inference
 - **Mistral**: European AI, EU data residency
3. **Free & Open Source Providers** (6 providers)
 - **XAI (Grok)**: Free tier, Twitter/X data access
 - **OpenRouter**: 100+ models, free options
 - **GitHub Copilot**: Free with subscription
 - **Qwen**: 2,000 free requests/day
 - **Ollama**: 100% local, privacy-focused
 - **Llama.cpp**: Direct inference, GGUF support

Part IV: Core Tools

8-13. **Six Tool Categories** (detailed implementations) - File System Tools - Shell Execution - Browser Automation - Web Tools - Voice-to-Code - Codebase Mapping

Part V: Advanced Workflows

14-21. **Eight Advanced Workflows** - Plan Mode (two-phase planning) - Multi-File Editing (atomic transactions) - Git Auto-Commit (semantic messages) - Context Compression (token optimization) - Tool Confirmation (interactive approval) - Checkpoint Snapshots (git-based rollback) - Autonomy Modes (5 levels of automation) - Vision Auto-Switch (automatic model switching)

Part VI: Distributed Computing

22-23. **Distributed Features** - Distributed Worker Setup (SSH, auto-installation) - Task Management (types, priorities, checkpointing)

Part VII: MCP Protocol

1. MCP Integration

- Model Context Protocol
- Transport Methods
- Tool Registration
- Custom MCP Servers

Part VIII: Multi-Client Usage

25-28. **Four Client Types** - CLI Client - Terminal UI - Desktop Application - Mobile Clients (iOS/Android)

Part IX: Security & Compliance

1. Security Best Practices

- Authentication & Authorization
- API Key Management
- Network Security
- Data Privacy
- Audit Logging

Part X: Troubleshooting & Reference

30-33. **Reference Materials** - Troubleshooting Guide - CLI Command Reference - API Reference - FAQ (10+ common questions)

Part XI: Advanced Use Cases

1. Advanced Use Cases

- Links to 8 detailed tutorials
-

Tutorial Summaries

Tutorial 1: Building a Web Application from Scratch

Duration: 45-60 minutes | **Lines:** 1,245

Complete step-by-step guide to building a Task Management REST API: - Project initialization with Plan Mode - Database schema design (PostgreSQL) - Three-layer architecture (handler/service/repository) - JWT authentication - CRUD operations - Unit testing - Docker deployment - Git auto-commit integration

Technologies: Go, Gin, PostgreSQL, JWT, Docker

Key HelixCode Features: - Plan Mode for structured development - Code generation with Claude 3.5 Sonnet - Auto-commit with semantic messages - Complete working application in <1 hour

Tutorial 2: Refactoring a Large Codebase

Duration: 30-45 minutes | **Lines:** 133

Learn to refactor legacy code with advanced HelixCode features: - Codebase mapping with Tree-sitter (247 files, 45K LOC) - Full context analysis with Gemini 2.5 Pro - Multi-file atomic edits with transactions - Checkpoint snapshots for safety - Context compression for long sessions - Iterative refactoring workflow

Results: 95% test coverage, modular architecture, 30 min vs. 2-3 days manual

Tutorial 3: Using Multiple AI Providers

Duration: 20 minutes | **Lines:** 110

Strategic provider selection for optimal results: - Provider selection guide by task type - Architecture design with Claude 4 Sonnet - Full codebase analysis with Gemini 2.5 Pro - Rapid prototyping with Groq - Privacy-sensitive code with Ollama - Automatic fallback configuration - Cost optimization strategies

Outcome: 70% cost reduction with strategic provider use

Tutorial 4: Browser Automation for Testing

Duration: 30 minutes | **Lines:** 149

Automate web testing with browser control: - Chrome/Chromium automation - E2E test generation - Form filling and interaction - Screenshot capture - Visual regression testing - Web scraping examples - CI integration

Technologies: Chromium, Playwright-style API

Tutorial 5: Voice-to-Code Workflow

Duration: 15 minutes | **Lines:** 118

Hands-free coding with voice input: - Whisper transcription setup - Voice command execution - Dictated code generation - Voice-controlled refactoring - Code review by voice - Text-to-speech responses - Accessibility features

Use Cases: Accessibility, multitasking, brainstorming

Tutorial 6: Multi-File Atomic Edits

Duration: 20 minutes | **Lines:** 134

Transaction-based editing across multiple files: - Transaction lifecycle - Cross-file refactoring (23 files, 47 changes) - Automatic rollback on test failure - Conflict detection - AI-powered multi-file refactoring - Conditional edits with code

Safety: All-or-nothing commits with full rollback

Tutorial 7: Distributed Development with Workers

Duration: 30 minutes | **Lines:** 165

Scale development with distributed SSH workers: - Worker setup (3 machines) - Auto-installation of HelixCode CLI - Hardware detection (CPU, RAM, GPU) - Task distribution and orchestration - Health monitoring dashboard - GPU acceleration for AI inference - Distributed test execution (3x speedup)

Infrastructure: SSH-based, supports mixed architectures

Tutorial 8: Using Plan Mode for Complex Projects

Duration: 45 minutes | **Lines:** 301

Master Plan Mode for large projects: - Complex project planning (real-time chat app) - Option generation (5 architecture options) - Phase-by-phase execution with approval - Iterative refinement during execution - Progress tracking - Pause/resume functionality - Auto-generated documentation - Custom plan templates

Results: 15K LOC project, 9 hours with AI vs. weeks manual

Tutorial 9: Creating a Custom LLM Provider

Duration: 30 minutes | **Lines:** 79

Extend HelixCode with a custom LLM provider: - Implement the `Provider` interface - `Provider` scaffold with `Name()`, `Generate()`, `GenerateStream()` - Configuration in `config/config.yaml` - Registration in provider registry via `init()` - Unit testing with Go's testing package

Tutorial 10: Adding a New Tool via MCP SDK

Duration: 30 minutes | **Lines:** 80

Create a custom tool for HelixCode's MCP server: - Define tool schema with JSON-RPC - Implement `Execute()` for business logic - Register tool in MCP registry - Test via WebSocket connection - Optional `Confirmable` interface for dangerous tools

Configuration Examples

1. `basic_config.yaml` (57 lines)

Purpose: Development and small projects

Features: - Minimal dependencies (Ollama for local AI) - Redis disabled (optional) - Debug logging - Development-friendly settings - Local file operations only

Best For: Learning, prototyping, single-developer projects

2. `provider_configs.yaml` (174 lines)

Purpose: Complete reference for all AI providers

Includes: - **8 Premium Providers:** Anthropic, Gemini, OpenAI, Bedrock, Azure, VertexAI, Groq, Mistral - **6 Free Providers:** XAI, OpenRouter, Copilot, Qwen, Ollama, Llama.cpp - Configuration examples for each - API key setup - Advanced features (caching, safety, etc.) - Model selection

Best For: Reference guide, multi-provider setups

3. `enterprise_setup.yaml` (248 lines)

Purpose: Production enterprise deployment

Features: - High availability (replicas, sentinels) - Security (SSO, encryption, audit logging) - Monitoring (Prometheus, Grafana, alerts) - Auto-scaling - Rate limiting and CORS - Cost management and budgets - Backup and disaster recovery - Compliance (GDPR, SOX, HIPAA)

Best For: Large organizations, mission-critical deployments

4. `multi_worker_setup.yaml` (261 lines)

Purpose: Distributed worker orchestration

Features: - **3 Worker Pools:** Build farm, GPU cluster, Test runners - **10+ Workers:** Mixed architectures (x86, ARM, macOS) - GPU workers (RTX 4090, A100) - Capability-based routing - Auto-scaling with cloud integration (AWS, GCP, Azure) - Cost optimization (spot instances, idle shutdown) - Comprehensive monitoring

Best For: Large-scale parallel development, CI/CD, ML workloads

Coverage Summary

LLM Providers: 14+

✓ Anthropic Claude (4 models) ✓ Google Gemini (5 models) ✓ OpenAI (6 models) ✓ AWS Bedrock (multi-model) ✓ Azure OpenAI ✓ Google VertexAI ✓ Groq (4 models) ✓ Mistral (4 models) ✓ XAI Grok (3 models) ✓ OpenRouter (100+ models) ✓ GitHub Copilot (5 models) ✓ Qwen (3 models) ✓ Ollama (any GGUF model) ✓ Llama.cpp (any GGUF model)

Core Tools: 6

✓ File System Tools (read, write, edit, search) ✓ Shell Execution (sandboxed, secure) ✓ Browser Automation (Chrome/Chromium) ✓ Web Tools (search, fetch, parse) ✓ Voice-to-Code (Whisper transcription) ✓ Codebase Mapping (Tree-sitter, 30+ languages)

Advanced Workflows: 8

✓ Plan Mode (two-phase planning) ✓ Multi-File Editing (atomic transactions) ✓ Git Auto-Commit (semantic messages) ✓ Context Compression (3 strategies) ✓ Tool Confirmation (risk-based approval) ✓ Checkpoint Snapshots (git-based) ✓ Autonomy Modes (5 levels) ✓ Vision Auto-Switch (automatic model selection)

Distributed Features: 2

✓ SSH Worker Setup (auto-installation) ✓ Task Management (distributed orchestration)

MCP Protocol: 1

✓ Full Model Context Protocol implementation

Clients: 4

✓ CLI Client ✓ Terminal UI (TUI) ✓ Desktop Application ✓ Mobile (iOS/Android)

Tutorials: 10

✓ Building Web Applications ✓ Refactoring Large Codebases ✓ Multiple AI Providers ✓ Browser Automation ✓ Voice-to-Code ✓ Multi-File Atomic Edits ✓ Distributed Development ✓ Plan Mode Mastery ✓ Custom LLM Provider ✓ Tool via MCP SDK

Configuration Examples: 4

✓ Basic Development Setup ✓ All Provider Configurations ✓ Enterprise Production Setup ✓ Multi-Worker Distributed Setup

Documentation Quality Metrics

Completeness

- **Total Lines:** 6,494
- **Main Manual:** 3,240 lines (exceeds 2,000 line requirement)
- **Tutorials:** 10 comprehensive guides
- **Examples:** 4 production-ready configurations
- **Coverage:** All 14+ providers, all tools, all workflows

Structure

- **Table of Contents:** Comprehensive, multi-level
- **Navigation:** Clear section numbering
- **Code Examples:** 100+ code snippets
- **Command Examples:** 200+ CLI commands
- **Configuration Examples:** 740 lines of YAML

Practical Value

- **Step-by-Step Guides:** All tutorials are actionable
- **Real-World Examples:** Task API, chat app, refactoring
- **Production-Ready Configs:** Enterprise and distributed setups
- **Best Practices:** Throughout all sections
- **Troubleshooting:** Common issues and solutions

Accessibility

- **Skill Levels:** Beginner to Advanced
 - **Learning Path:** Progressive difficulty
 - **Quick Reference:** Tables, lists, summaries
 - **Search-Friendly:** Clear headings, keywords
-

Usage Recommendations

For Beginners

1. Start with **Main README** sections 1-3 (Introduction, Installation, First Project)
2. Follow **Tutorial 1** (Building Web App)
3. Explore **Tutorial 3** (Multiple AI Providers)
4. Use **basic_config.yaml** as template

For Intermediate Developers

1. Review **Part III** (LLM Providers) to choose optimal provider
2. Complete **Tutorial 2** (Refactoring)
3. Try **Tutorial 6** (Multi-File Edits)
4. Experiment with **Tutorial 4** (Browser Automation)

For Advanced Users

1. Study **Part V** (Advanced Workflows)
2. Master **Tutorial 8** (Plan Mode)
3. Setup **Tutorial 7** (Distributed Workers)
4. Deploy with **enterprise_setup.yaml**

For Enterprise Teams

1. Review **Part IX** (Security Best Practices)
2. Use **enterprise_setup.yaml** as base
3. Configure **multi_worker_setup.yaml** for scale
4. Implement **Tutorial 7** (Distributed Development)

Key Features Highlighted

Advanced AI Capabilities

- **Extended Thinking** (Claude 4): Automatic reasoning mode
- **Prompt Caching** (Claude): 90% cost reduction
- **2M Token Context** (Gemini 2.5 Pro): Process entire codebases
- **Ultra-Fast Inference** (Groq): 500+ tokens/second
- **Local Models** (Ollama/Llama.cpp): 100% privacy

Intelligent Workflows

- **Plan Mode**: Two-phase plan-then-execute with options
- **Multi-File Edits**: Atomic transactions with rollback
- **Auto-Commit**: LLM-generated semantic messages
- **Context Compression**: Automatic conversation summarization
- **Snapshots**: Git-based checkpoint system

Enterprise Features

- **Distributed Workers**: SSH-based parallel execution
- **Auto-Scaling**: Cloud integration (AWS/GCP/Azure)
- **High Availability**: Replicas, sentinels, load balancing
- **Security**: SSO, encryption, audit logging, compliance
- **Monitoring**: Prometheus, Grafana, alerts

Developer Experience

- **Voice-to-Code**: Hands-free development
 - **Browser Automation**: E2E testing and scraping
 - **Codebase Mapping**: Tree-sitter AST analysis
 - **Multi-Client**: CLI, TUI, Desktop, Mobile
 - **MCP Protocol**: Standard AI tool integration
-

Future Enhancements

Planned Additions

- ☐ Video tutorials (screencasts)
- ☐ Architecture diagrams (in images/)
- ☐ API reference documentation
- ☐ Migration guides
- ☐ Contributing guidelines
- ☐ Plugin development guide

Suggested Improvements

- ☐ Interactive examples (CodePen/CodeSandbox)
- ☐ Comparison with competitors
- ☐ Case studies from real users
- ☐ Performance benchmarks
- ☐ Cost calculators

Maintenance Notes

Update Frequency

- **Quarterly:** Provider updates (new models, pricing)
- **Monthly:** Tutorial refinements
- **As Needed:** Configuration examples

Version Control

- All documentation versioned with main codebase
- Changes tracked in git commit history
- Major updates receive version bumps

Community Contributions

- Open to pull requests
- Follow existing structure and style

- Include code examples
 - Test all commands and configurations
-

Contact & Support

Documentation Issues

- GitHub Issues: <https://github.com/your-org/helixcode/issues>
- Label: `documentation`

Questions

- Community Forum: <https://community.helixcode.dev>
- Discord: <https://discord.gg/helixcode>
- Email: docs@helixcode.dev

Contributing

- See CONTRIBUTING.md
 - Style guide: docs/STYLE_GUIDE.md
 - Templates: .github/ISSUE_TEMPLATE/
-

Acknowledgments

Technologies Referenced

- **AI Providers:** Anthropic, Google, OpenAI, AWS, Azure, Groq, Mistral, XAI, OpenRouter, GitHub, Qwen, Ollama, Llama.cpp
- **Frameworks:** Gin, pgx, JWT, Docker, Kubernetes
- **Tools:** PostgreSQL, Redis, Prometheus, Grafana
- **Protocols:** MCP (Model Context Protocol)

Inspiration

- Claude Code (Anthropic's official CLI)
 - Codename Goose (Rust AI agent)
 - OpenCode (Go terminal agent)
 - Qwen Code (TypeScript agent)
 - Cursor IDE
 - GitHub Copilot
-

Documentation Created: November 6, 2025 **Last Updated:** May 8, 2026 **Total Lines:** 6,494 lines **Status:** Complete  **Version:** 2.1

Sources verified 2026-05-29: project go.mod + CLAUDE.md

§3.1 (creation-summary doc; no third-party operator instructions)

Reviewed against Constitution §11.4.99. This is a meta “creation summary” of the User Manual — it inventories line counts, tutorial titles, and config files; it contains **no standalone third-party setup instructions** to cross-reference against a vendor doc. Notes:

- **Provider / model references** (e.g. “Claude 3.5 Sonnet”, “Gemini 2.5 Pro”, “GPT-4.1, O1/O3”, “Groq 500+ tok/s”, per-provider model counts) are **LLMsVerifier-runtime-sourced per CONST-036** — descriptive/illustrative and may be stale. Authoritative live list: `helixcode llm models list`. Not re-pinned (LLMsVerifier is the single source of truth).
- **Frameworks/tools named** (Gin, pgx, JWT, Docker, Kubernetes, PostgreSQL, Redis, Prometheus, Grafana, MCP) appear as acknowledgements only, with **no version pins** in this file. Repo version authority is `go.mod + CLAUDE.md §3.1` (Go 1.26 / go1.26.3 latest, PostgreSQL 15+, Redis 7+ — all confirmed current this run).
- **External links** (`github.com/your-org/helixcode`, `community.helixcode.dev`, `discord.gg/helixcode`, `docs@helixcode.dev`) are **placeholder URLs**, not live HelixDevelopment endpoints. Negative finding: left as-is, flagged for a future docs pass (out of scope for this §11.4.99 batch).
- The hardcoded macOS path in “Directory Structure Created” (`/Users/milosvasic/Projects/helix_code/...`) is a stale absolute path; flagged, not in scope for this footer batch.